



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Práctica Guiada 2: Arquitectura Empresarial con Spring Boot, Frontend Modular con Web Components, Maquetación Elástica y Depuración Multi-Target en VS Code

Resumen

Esta práctica guiada tiene como objetivo introducir al estudiantado en el desarrollo de software empresarial full-stack partiendo desde cero. En esta práctica, usted construirá una aplicación de **Panel de Control del Estudiante** estructurada sobre el backend mediante **Spring Boot**, el framework estándar de la industria para desarrollo de APIs en Java.

En la capa del cliente, se implementará un frontend interactivo utilizando **Web Components** nativos del navegador (Custom Elements, Shadow DOM y HTML Templates) y técnicas de diseño

responsivo avanzadas como **CSS Grid elástico**, **Custom Properties (Variables)**, **Nesting nativo** y **Container Queries**. Para la automatización del proyecto y el ciclo de vida se empleará **Maven**, y para el control de versiones profesional se inicializará un repositorio en **Git y GitHub** estructurado por ramas. Finalmente, se configurará una sesión de **Depuración Multi-Target en VS Code** para interceptar y corregir fallas simultáneamente en el backend y el frontend.

Conceptos Principales (El 'Qué')

1. Spring Boot y Controladores REST

Spring Boot es una extensión del ecosistema de Spring que simplifica radicalmente el desarrollo de aplicaciones Java empresariales. A través del principio de "configuración sobre convención", elimina la necesidad de configurar manualmente servidores de aplicaciones web gracias a su servidor **Tomcat embebido**. Las peticiones HTTP se gestionan mediante **Controladores REST** (`@RestController`), los cuales mapean URLs a métodos de Java utilizando anotaciones (ej. `@GetMapping`, `@PostMapping`) y serializan automáticamente los objetos de retorno en formato JSON gracias al componente integrado Jackson.

2. Web Components Nativo (Componentización Web)

En lugar de depender de librerías pesadas, los navegadores modernos admiten la creación de elementos de interfaz de usuario encapsulados y reutilizables mediante tres tecnologías del consorcio W3C:

- * **Custom Elements (Elementos Personalizados)**: Permite registrar nuevas etiquetas HTML heredando de la clase de JavaScript `HTMLElement`.
- * **Shadow DOM (DOM en la Sombra)**: Proporciona un aislamiento completo (encapsulamiento). Los estilos y el árbol HTML interno de un componente no sufren interferencias del CSS o JS global del documento, y viceversa.
- * **HTML Templates & Slots**: La etiqueta `<template>` define estructuras HTML inactivas en memoria que se clonan eficientemente para cada nueva instancia del componente. Los `<slot>` actúan como puntos de inserción dinámicos para inyectar contenido externo desde el DOM principal.
- * **Atributos data-* y la API .dataset**: Permiten enviar parámetros personalizados desde las etiquetas HTML al código JavaScript del componente.

3. CSS Moderno: Custom Properties, Nesting y Container Queries

- **Custom Properties (Variables CSS)**: Almacenan valores estéticos dinámicos en cascada (`--mi-color: #3b82f6;`) que pueden redefinirse en tiempo de ejecución o cambiarse

mediante JavaScript.

- **Nesting Nativo:** Permite anidar selectores de CSS unos dentro de otros (`.tarjeta { h3 { ... } }`) reduciendo la redundancia de código y mejorando la legibilidad.
- **CSS Grid Elástico:** La regla `repeat(auto-fit, minmax(300px, 1fr))` instruye al navegador a ajustar automáticamente el número de columnas de una cuadrícula en función del espacio físico de la pantalla, sin usar Media Queries rígidas.
- **Container Queries (@container):** A diferencia de las Media Queries (que responden al tamaño de la pantalla global del viewport), las Container Queries responden al ancho del **contenedor inmediato** que aloja al componente. Esto permite que una tarjeta se organice verticalmente en una barra lateral estrecha, u horizontalmente en un panel central ancho, independientemente de la resolución de la pantalla.

4. Depuración Multi-Target (Concurrent Debugging)

En sistemas de pila completa (*full-stack*), un flujo de datos erróneo puede originarse tanto en el cliente como en el servidor. La depuración multi-target permite a Visual Studio Code conectar de manera paralela el depurador de Java al servidor Spring Boot y el depurador de Chrome a la sesión del navegador. Esto permite rastrear el flujo de una variable desde que el usuario hace clic en la pantalla hasta que impacta la base de datos o el controlador del backend.

Analogía (La Intuición)

- **Spring Boot:** Imagine que para abrir un restaurante, en lugar de comprar los ladrillos, cemento y tuberías para edificar el local (Java nativo), usted adquiere un contenedor móvil prefabricado que ya incluye cocina industrial, tuberías listas, aire acondicionado y chef (Tomcat embebido). Solo debe conectar la electricidad (Maven) y empezar a preparar su menú (Controladores).
- **Web Components & Shadow DOM:** Imagine un **reproductor de DVD físico**. La carcasa externa es el Custom Element (`<reproductor-dvd>`). El cableado eléctrico e interno y los motores láser que lo hacen funcionar están encapsulados herméticamente (Shadow DOM); la pintura de su sala no afecta el color interno de los circuitos. El compartimento de entrada para el disco es el **Slot**, donde usted inyecta el contenido dinámico (la película) desde el exterior sin alterar el funcionamiento interno del reproductor.

- **Container Queries:** Imagine un **organizador elástico de equipaje**. Si usted coloca el organizador dentro de una maleta pequeña de mano (contenedor estrecho), los compartimentos del organizador se apilan de forma compacta en una sola columna vertical. Si saca el mismo organizador y lo coloca en un maletín de viaje grande (contenedor ancho), el organizador se despliega horizontalmente en tres columnas. El organizador no sabe cuán grande es el avión o la habitación; solo se adapta al espacio exacto de la maleta que lo contiene.

Ejemplo de Código e Implementación (El 'Cómo')

A continuación, crearemos un proyecto completo desde cero, configurando el repositorio local de Git, estructurando Spring Boot, diseñando el frontend interactivo y configurando las herramientas profesionales de desarrollo en VS Code.

Estándar de Nomenclatura del Curso:

Recuerde que los nombres de los paquetes Java deben seguir obligatoriamente la estructura estándar del curso:

```
cr.ac.ucr.paraíso.ie.<carnet>.práctica2
```

Donde `<carnet>` corresponde a su identificador universitario en minúsculas (ej. `b98765`). En los siguientes pasos, reemplace la palabra `carnet` con sus datos reales.

Paso 1: Configuración Inicial de Git Local

Antes de escribir código, configure su repositorio local de Git para realizar el seguimiento del desarrollo desde el primer instante:

1. Abra una terminal en su directorio de trabajo, cree la carpeta de espacio de trabajo general para el curso y acceda a ella. Posteriormente, cree la carpeta de esta práctica y acceda a ella:

```
# Crear espacio de trabajo general y acceder a él
mkdir DSW4_workspace
cd DSW4_workspace

# Crear el directorio del proyecto de la Práctica 2 y acceder
```

```
mkdir practica-2
cd practica-2
```

2. Configure sus credenciales globales (reemplace por sus datos reales de GitHub):

```
git config --global user.name "Su Nombre"
git config --global user.email "su-correo@mail.com"
```

3. Inicialice el repositorio Git en la carpeta raíz del proyecto:

```
git init
```

4. Verificación y Configuración de la Rama Principal (main):

Las versiones anteriores de Git inicializan la rama principal como `master` por defecto. GitHub y las prácticas de desarrollo actuales utilizan `main` de manera estandarizada. Para prevenir conflictos de ramas al realizar el `push` remoto: * Consulte el nombre de la rama actual ejecutando: `bash git status` * Si la terminal indica que se encuentra en la rama `master`, cámbiele el nombre a `main` ejecutando: `bash git branch -M main` * (Opcional) Configure su cliente de Git de forma global para que toda inicialización futura cree por defecto la rama `main`: `bash git config --global init.defaultBranch main`

5. Configure el archivo `.gitignore` en la raíz del proyecto para omitir archivos compilados, temporales o de configuración del entorno de desarrollo. Cree un archivo llamado `.gitignore` y agregue el siguiente contenido:

```
/target/
/.settings/
/.classpath
/.project
/.idea/
/*.iml
.DS_Store
```

6. Verifique el estado inicial en Git. Al abrir la pestaña de **Source Control** en VS Code (atajo `ctrl + Shift + G`), verá los archivos no rastreados. Guarde este estado inicial en su primer commit:

```
git add .gitignore
git commit -m "chore: inicializar archivo gitignore"
```

Paso 2: Configuración de Maven y Estructura Spring Boot (Backend)

1. Cree el archivo de gestión de dependencias pom.xml ubicado en la carpeta backend/pom.xml. Este archivo configura las dependencias de Spring Web y DevTools:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.1.5</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>

  <groupId>cr.ac.ucr.paraíso.ie.carnet.practica2</groupId>
  <artifactId>practica-2</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <!-- NOTA: Reemplace 21 por la versión exacta -->
    <!-- de Java instalada en su PC (ej: 17 o 21) -->
    <java.version>21</java.version>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <!-- Starter de Spring Boot para desarrollo Web -->
    <!-- Incluye REST, MVC y Tomcat embebido -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- DevTools para reinicio rápido en cambios de código -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-devtools</artifactId>
```

```

        <optional>true</optional>
    </dependency>
</dependencies>
</parent>

<build>
    <plugins>
        <!-- Plugin oficial de Spring Boot para empaquetado ejecutable -->
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>

```

2. Cree la estructura de carpetas de Spring Boot para las clases Java y los recursos del frontend. Para lograr un diseño ordenado y extensible, el backend seguirá una arquitectura estándar de capas.

Paquetes requeridos: `data` (persistencia), `business` (servicios), `controller` (controladores REST), `domain` (modelos).

Ejecute los siguientes comandos en la terminal de VSC (reemplazando carnet por el suyo):

```

# Crear carpetas de los paquetes requeridos
mkdir backend/src/main/java/cr/ac/ucr/paraíso/ie/carnet/practica2/domain
mkdir backend/src/main/java/cr/ac/ucr/paraíso/ie/carnet/practica2/controller
mkdir backend/src/main/java/cr/ac/ucr/paraíso/ie/carnet/practica2/business
mkdir backend/src/main/java/cr/ac/ucr/paraíso/ie/carnet/practica2/data

# Crear carpeta estándar para servir archivos estáticos del frontend
mkdir backend/src/main/resources/static

```

3. Cree la clase principal de arranque `Practica2Application.java` en la ruta:

`backend/src/main/java/cr/ac/ucr/paraíso/ie/carnet/practica2/Practica2Application.java`

```

package cr.ac.ucr.paraíso.ie.carnet.practica2;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

```

```

public class Practica2Application {
    public static void main(String[] args) {
        // Inicializa Spring Boot y levanta Tomcat en puerto 8080
        SpringApplication.run(Practica2Application.class, args);
    }
}

```

4. Cree el modelo de datos `Asignacion.java` dentro del paquete `domain`. Ubíquelo en:
`backend/src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica2/domain/Asignacion.java`

```

package cr.ac.ucr.paraiso.ie.carnet.practica2.domain;

public class Asignacion {
    private int id;
    private String titulo;
    private String descripcion;
    private String fechaEntrega;
    private boolean completada;

    // Constructor vacío requerido para la deserialización JSON
    public Asignacion() {}

    public Asignacion(int id, String titulo, String descripcion,
        String fechaEntrega, boolean completada) {
        this.id = id;
        this.titulo = titulo;
        this.descripcion = descripcion;
        this.fechaEntrega = fechaEntrega;
        this.completada = completada;
    }

    // Getters y Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }

    public String getDescripcion() { return descripcion; }
    public void setDescripcion(String descripcion) {
        this.descripcion = descripcion;
    }

    public String getFechaEntrega() { return fechaEntrega; }

```



```

    public void setFechaEntrega(String fechaEntrega) {
        this.fechaEntrega = fechaEntrega;
    }

    public boolean isCompletada() { return completada; }
    public void setCompletada(boolean completada) {
        this.completada = completada;
    }
}

```

5. Cree el controlador REST `AsignacionController.java` dentro del paquete `controller` para exponer la API. Ubíquelo en:

`backend/src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica2/controller/AsignacionController.java`

```

package cr.ac.ucr.paraiso.ie.carnet.practica2.controller;

import cr.ac.ucr.paraiso.ie.carnet.practica2.domain.Asignacion;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.ArrayList;
import java.util.List;

@RestController
@RequestMapping("/api/asignaciones")
@CrossOrigin(origins = "*")
// Habilitar peticiones desde cualquier origen para evitar CORS en desarrollo
public class AsignacionController {

    private final List<Asignacion> asignaciones = new ArrayList<>();
    private int autoincrementId = 1;

    public AsignacionController() {
        // Datos semilla iniciales
        asignaciones.add(new Asignacion(
            autoincrementId++,
            "Práctica Guiada 1",
            "Completar el servidor Java nativo",
            "2026-08-25",
            true
        ));
        asignaciones.add(new Asignacion(
            autoincrementId++,
            "Laboratorio 1",

```

```

        "Implementar la persistencia con Maven",
        "2026-08-30",
        false
    ));
}

@GetMapping
public List<Asignacion> obtenerTodas() {
    System.out.println("[API GET] Listado solicitado...");
    return asignaciones;
}

@PostMapping
public ResponseEntity<?> agregarAsignacion(@RequestBody Asignacion nueva) {
    System.out.println("[API POST] Recibiendo: " + nueva.getTitulo());

    // ERROR INTENCIONAL PARA EL TALLER DE DEPURACIÓN:
    // Si el título es nulo/vacío, simulamos un NullPointerException
    if (nueva.getTitulo() == null || nueva.getTitulo().trim().isEmpty()) {
        throw new NullPointerException(
            "El título no puede procesarse porque es nulo o vacío."
        );
    }

    // Si la descripción está vacía, retornamos BAD_REQUEST
    if (nueva.getDescripcion() == null ||
        nueva.getDescripcion().trim().isEmpty()) {
        return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body("La descripción es obligatoria.");
    }

    nueva.setId(autoincrementId++);
    asignaciones.add(nueva);
    return ResponseEntity.status(HttpStatus.CREATED).body(nueva);
}
}

```

Paso 3: Creación de la Interfaz y Web Components (Frontend)

Todo el código de nuestra interfaz de usuario se ubicará en la carpeta de recursos estáticos de Spring Boot (backend/src/main/resources/static/).

1. Cree el archivo HTML principal index.html en la ruta:

backend/src/main/resources/static/index.html. Asegure un anidamiento de etiquetas de 4 espacios:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Panel de Control del Estudiante</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="app-header">
    <h1>UCR - Portal Académico</h1>
    <p>Curso: IF0009 - Desarrollo de Software IV</p>
  </header>
  <main class="app-container">
    <!-- Formulario para agregar nuevas asignaciones -->
    <section class="form-section">
      <h2>Registrar Nueva Asignación</h2>
      <form id="formAsignacion">
        <div class="form-group">
          <label for="titulo">Título de la Tarea:</label>
          <input type="text" id="titulo"
            placeholder="Ej: Laboratorio 2 Spring Boot">
        </div>
        <div class="form-group">
          <label for="descripcion">Descripción:</label>
          <textarea id="descripcion" rows="3"
            placeholder="Detalle instrucciones..."></textarea>
        </div>
        <div class="form-group">
          <label for="fecha">Fecha de Entrega:</label>
          <input type="date" id="fecha">
        </div>
        <button type="submit" class="btn-submit">
          Guardar Asignación
        </button>
      </form>
    </section>
  </main>
</body>
</html>
```

```

        </form>
    <!-- Contenedor para notificar errores o éxitos del API -->
    <div id="notificacion" class="notificacion hidden"></div>
</section>

<!-- Componente Contenedor del Dashboard -->
<section class="dashboard-section">
    <h2>Listado de Entregas Pendientes</h2>
    <!-- Usamos nuestro Custom Element del Dashboard -->
    <panel-estudiante id="panelDashboard"></panel-estudiante>
</section>

</main>

<!-- Importación de los Web Components como módulos de JS -->
<script src="components/TarjetaAsignacion.js" type="module"></script>
<script src="components/PanelEstudiante.js" type="module"></script>
</body>
</html>

```

2. Cree la carpeta para los componentes JavaScript del frontend:

```
mkdir backend/src/main/resources/static/components
```

3. Cree la lógica del Custom Element para la tarjeta de asignación en `TarjetaAsignacion.js` utilizando **Shadow DOM**. Ubíquelo en:

`backend/src/main/resources/static/components/TarjetaAsignacion.js`

```

const template = document.createElement('template');
template.innerHTML = `
    <style>
        :host {
            display: block;
            container-type: inline-size;
            width: 100%;
        }

        .tarjeta {
            background-color: var(--card-bg, #ffffff);
            border: 1px solid var(--border-color, #e2e8f0);
            border-radius: 8px;
            padding: 18px;
        }
    </style>
`

```

```

        transition: transform 0.2s, border-color 0.2s;
        display: flex;
        flex-direction: column;
        gap: 8px;
        box-shadow: 0 2px 4px rgba(0, 0, 0, 0.05);
    }

    .tarjeta:hover {
        border-color: var(--primary-color, #3b82f6);
        transform: translateY(-2px);
    }

    .estado {
        display: inline-block;
        align-self: flex-start;
        padding: 4px 8px;
        border-radius: 4px;
        font-size: 0.75rem;
        font-weight: bold;
        text-transform: uppercase;
    }

    .pendiente {
        background-color: #fee2e2;
        color: #ef4444;
    }

    .completada {
        background-color: #dcfce7;
        color: #22c55e;
    }

    h3 {
        margin: 0;
        color: var(--text-dark, #1e293b);
        font-size: 1.15rem;
    }

    p {
        margin: 0;
        color: var(--text-muted, #64748b);
        font-size: 0.9rem;
        line-height: 1.4;
    }

    .meta {

```

```

        font-size: 0.8rem;
        color: var(--text-muted, #64748b);
        border-top: 1px solid var(--border-color, #e2e8f0);
        padding-top: 8px;
        margin-top: 4px;
    }

    /* ----- */
    /* CONTAINER QUERY: Adaptar la tarjeta si contenedor > 400px */
    /* Hace el componente modular e independiente del viewport */
    @container (min-width: 400px) {
        .tarjeta {
            flex-direction: row;
            align-items: center;
            justify-content: space-between;
            gap: 20px;
        }

        .cuerpo-tarjeta {
            flex: 2;
        }

        .info-derecha {
            flex: 1;
            display: flex;
            flex-direction: column;
            align-items: flex-end;
            text-align: right;
            gap: 6px;
        }

        .meta {
            border-top: none;
            padding-top: 0;
            margin-top: 0;
        }
    }
}
</style>

<div class="tarjeta">
    <div class="cuerpo-tarjeta">
        <span class="estado" id="insigniaEstado">Pendiente</span>
        <h3><slot name="titulo">Sin título</slot></h3>
        <p><slot name="descripcion">Sin descripción</slot></p>
    </div>
    <div class="info-derecha">

```

```

        <div class="meta">
            Entrega: <span id="fechaEntrega">-</span>
        </div>
    </div>
</div>
`;
export class TarjetaAsignacion extends HTMLElement {
    constructor() {
        super();
        // 1. Shadow Root abierto para encapsular marcado y estilos
        this.attachShadow({ mode: 'open' });

        // 2. Clonar y adjuntar el nodo al Shadow DOM
        this.shadowRoot.appendChild(template.content.cloneNode(true));
    }

    connectedCallback() {
        this.actualizarAtributos();
    }

    actualizarAtributos() {
        // BUG INTENCIONAL PARA EL TALLER DE DEPURACIÓN (Paso 4):
        // Intentaremos leer data-fecha-entrega pero en JS
        // procesaremos un atributo no coincidente
        const fecha = this.dataset.fechaEntrega || 'Sin fecha';
        const completada = this.dataset.completada === 'true';

        const insignia = this.shadowRoot.getElementById('insigniaEstado');
        const campoFecha = this.shadowRoot.getElementById('fechaEntrega');

        campoFecha.textContent = fecha;

        if (completada) {
            insignia.textContent = 'Completada';
            insignia.className = 'estado completada';
        } else {
            insignia.textContent = 'Pendiente';
            insignia.className = 'estado pendiente';
        }
    }
}

customElements.define('tarjeta-asignacion', TarjetaAsignacion);

```

4. Cree el Web Component contenedor del listado general `PanelEstudiante.js`. Este consumirá la API de Spring Boot y construirá la cuadrícula inyectando dinámicamente las tarjetas. Ubíquelo en: `backend/src/main/resources/static/components/PanelEstudiante.js`

```
export class PanelEstudiante extends HTMLElement {
  constructor() {
    super();
    // No usamos Shadow DOM para permitir recibir los estilos
    // CSS de la rejilla global de forma directa (Light DOM).
    this.innerHTML = `
      <div class="grid-asignaciones" id="gridAsignaciones">
        <!-- Tarjetas inyectadas por JavaScript -->
      </div>
    `;
  }

  connectedCallback() {
    this.cargarAsignaciones();
  }

  // GET asíncrono al backend de Spring Boot
  async cargarAsignaciones() {
    const grid = this.querySelector('#gridAsignaciones');
    grid.innerHTML = '<p class="cargando">Cargando tareas...</p>';

    try {
      const respuesta = await fetch('/api/asignaciones');
      if (!respuesta.ok) throw new Error('Error al conectar con la API.');

      const datos = await respuesta.json();
      grid.innerHTML = '';

      if (datos.length === 0) {
        grid.innerHTML = '<p class="vacio">No hay tareas.</p>';
        return;
      }

      // Recorrer e inyectar dinámicamente cada Web Component
      datos.forEach(asig => {
        const tarjeta = document.createElement('tarjeta-asignacion');

        // Pasar atributos de datos al componente
        tarjeta.setAttribute('data-fechaEntrega', asig.fechaEntrega);
        tarjeta.dataset.completada = asig.completada;
      });
    }
  }
}
```



```

        tarjeta.innerHTML = `
            <span slot="titulo">${asig.titulo}</span>
            <span slot="descripcion">${asig.descripcion}</span>
        `;

        grid.appendChild(tarjeta);
    });

    } catch (error) {
        console.error('Error cargando datos:', error);
        grid.innerHTML = `
            <p class="error-panel">
                Error: No se pudo conectar a Spring Boot.
            </p>
        `;
    }
}

customElements.define('panel-estudiante', PanelEstudiante);

```

5. Asocie el escuchador de envío del formulario en la parte inferior de `index.html` (antes del cierre de la etiqueta `</body>`):

```

<script type="module">
    const form = document.getElementById('formAsignacion');
    const notif = document.getElementById('notificacion');
    const dashboard = document.getElementById('panelDashboard');

    form.addEventListener('submit', async (e) => {
        e.preventDefault();

        const titulo = document.getElementById('titulo').value;
        const descripcion = document.getElementById('descripcion').value;
        const fecha = document.getElementById('fecha').value;

        notif.classList.add('hidden');

        const payload = {
            titulo: titulo,
            descripcion: descripcion,
            fechaEntrega: fecha,
            completada: false
        };
    });

```

```

    try {
        const response = await fetch('/api/asignaciones', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify(payload)
        });

        if (response.status === 201) {
            mostrarMensaje('¡Asignación guardada!', 'exito');
            form.reset();
            dashboard.cargarAsignaciones();
        } else {
            const errorMsg = await response.text();
            mostrarMensaje(
                `Error ${response.status}: ${errorMsg}`,
                'error'
            );
        }
    } catch (error) {
        mostrarMensaje(
            'Error de red: No se conectó al servidor.',
            'error'
        );
    }
});

function mostrarMensaje(texto, tipo) {
    notif.textContent = texto;
    notif.className = `notificacion ${tipo}`;
}
</script>

```

Paso 4: Maquetación y CSS Moderno

Cree la hoja de estilos en la ruta: `backend/src/main/resources/static/style.css`. Aplique variables CSS, anidamiento nativo y una paleta HSL premium:

```

/* 1. Declaración de variables (Tokens CSS) globales */
:root {

```

```

    --primary-color: hsl(220, 90%, 56%);
    --bg-light: hsl(210, 40%, 98%);
    --bg-dark: hsl(222, 47%, 11%);
    --bg-card: hsl(223, 47%, 16%);
    --text-main: hsl(210, 40%, 98%);
    --text-muted: hsl(215, 20%, 65%);
    --border-color: hsl(217, 32%, 22%);
    --success: hsl(142, 70%, 45%);
    --error-color: hsl(0, 84%, 60%);
    --font-stack: system-ui, -apple-system, sans-serif;
}

* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    background-color: var(--bg-dark);
    color: var(--text-main);
    font-family: var(--font-stack);
    padding: 20px;
    min-height: 100vh;
}

.app-header {
    text-align: center;
    margin-bottom: 40px;
    padding-bottom: 20px;
    border-bottom: 1px solid var(--border-color);

    h1 {
        color: var(--primary-color);
        font-size: 2rem;
        margin-bottom: 8px;
    }

    p {
        color: var(--text-muted);
        font-size: 0.95rem;
    }
}

.app-container {
    display: flex;

```

```

    flex-direction: column;
    gap: 30px;
    max-width: 1200px;
    margin: 0 auto;
}

/* En pantallas grandes, divide formulario y panel en dos columnas */
@media (min-width: 768px) {
    flex-direction: row;
    align-items: flex-start;
}
}

.form-section, .dashboard-section {
    background-color: var(--bg-card);
    border: 1px solid var(--border-color);
    border-radius: 12px;
    padding: 30px;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.3);
}

.form-section {
    flex: 1;
    position: sticky;
    top: 20px;
}

.dashboard-section {
    flex: 2;
}

h2 {
    color: var(--text-main);
    font-size: 1.4rem;
    margin-bottom: 20px;
    border-left: 4px solid var(--primary-color);
    padding-left: 10px;
}

/* 2. Nesting Nativo para elementos de formulario */
#formAsignacion {
    display: flex;
    flex-direction: column;
    gap: 15px;

    .form-group {
        display: flex;

```

```

    flex-direction: column;
    gap: 6px;

    label {
        font-size: 0.85rem;
        color: var(--text-muted);
        font-weight: bold;
    }

    input, textarea {
        padding: 10px;
        background-color: var(--bg-dark);
        border: 1px solid var(--border-color);
        border-radius: 6px;
        color: var(--text-main);
        font-size: 0.95rem;
        outline: none;
        transition: border-color 0.2s;

        &:focus {
            border-color: var(--primary-color);
        }
    }

    .btn-submit {
        padding: 12px;
        background-color: var(--primary-color);
        color: white;
        border: none;
        border-radius: 6px;
        font-weight: bold;
        cursor: pointer;
        transition: background-color 0.2s;

        &:hover {
            background-color: hsl(220, 90%, 48%);
        }
    }
}

/* 3. Rejilla CSS Grid elástica para desplegar las asignaciones */
.grid-asignaciones {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
    gap: 20px;

```

```

}
}
.notification {
  margin-top: 15px;
  padding: 10px;
  border-radius: 6px;
  font-size: 0.85rem;
  text-align: center;
  border: 1px solid transparent;

  &.exito {
    background-color: rgba(34, 197, 94, 0.1);
    border-color: var(--success);
    color: var(--success);
  }

  &.error {
    background-color: rgba(239, 68, 68, 0.1);
    border-color: var(--error-color);
    color: var(--error-color);
  }
}
}

.hidden {
  display: none !important;
}

.cargando, .vacio {
  grid-column: 1 / -1;
  text-align: center;
  color: var(--text-muted);
  padding: 40px 0;
}

```

Paso 5: Configuración de Depuración Multi-Target en VS Code (`launch.json`)

Para depurar el backend y el frontend simultáneamente, configure el archivo de lanzamiento:

1. En el directorio raíz del proyecto (un nivel arriba de `backend/`), cree una carpeta llamada `.vscode/`.
2. Dentro de esa carpeta, cree el archivo `launch.json` con la siguiente estructura:

```

{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "java",
      "name": "Debug Spring Boot Backend",
      "request": "launch",
      "mainClass": "cr.ac.ucr.paraíso.ie.carnet.practica2.Practica2Application",
      "projectName": "practica-2"
    },
    {
      "type": "chrome",
      "name": "Debug Frontend in Chrome",
      "url": "http://localhost:8080",
      "webRoot": "${workspaceFolder}/backend/src/main/resources/static",
      "request": "launch"
    }
  ],
  "compounds": [
    {
      "name": "Debug Full Stack (Java + Chrome)",
      "configurations": [
        "Debug Spring Boot Backend",
        "Debug Frontend in Chrome"
      ]
    }
  ]
}

```

Paso 6: Taller de Depuración Guiado (Cacería de Bugs)

Una vez estructurado el proyecto, realizaremos un ejercicio práctico en clase para interactuar con el depurador y diagnosticar fallas lógicas.

Ejercicio 1: Diagnóstico de Excepciones del Servidor (NPE en Backend)

1. En la barra lateral de VS Code, haga clic en el icono de **Run and Debug** (o presione **Ctrl + Shift + D**).
2. Elija la configuración compuesta **Debug Full Stack (Java + Chrome)** y presione **F5**. Esto iniciará el servidor e iniciará Chrome en **http://localhost:8080**.

3. Abra `AsignacionController.java` y coloque un punto de interrupción (círculo rojo) en la línea: `* if (nueva.getTitulo() == null || nueva.getTitulo().trim().isEmpty()) {`

4. En Chrome, deje el campo **Título de la Tarea** vacío, escriba una descripción y presione **Guardar Asignación**.

5. Observe que la página se congela y VS Code resalta la línea en amarillo.

6. Revise el panel **Variables** (sección **Local**). Expanda el objeto `nueva` para constatar que `titulo` es `null`.

7. Presione **F10** (Step Over) para avanzar. Verá que entra al bloque y arroja la excepción. Presione **F5** para reanudar.

Ejercicio 2: Diagnóstico de Errores de Lectura (Dataset en Frontend)

1. Note que las asignaciones por defecto muestran la fecha como `-` en pantalla.

2. Abra `TarjetaAsignacion.js` y coloque un punto de interrupción en: `* const fecha = this.dataset.fechaEntrega || 'Sin fecha';`

3. Recargue el navegador Chrome. La ejecución se pausará en dicha línea.

4. En el panel izquierdo de VS Code, expanda el objeto `this.dataset`. Verá que la propiedad se llama `fechaentrega` (todo en minúsculas).

5. **Causa:** En `PanelEstudiante.js`, el atributo se inyectó como:

```
tarjeta.setAttribute('data-fechaEntrega', asig.fechaEntrega);
```

El parser de HTML convierte los nombres de atributos a minúsculas, resultando en `data-fechaentrega` en lugar de la separación por guiones normalizada.

6. **Solución:** Modifique la línea en `TarjetaAsignacion.js` para leer la propiedad correcta de `dataset` en minúsculas: `javascript const fecha = this.dataset.fechaentrega || 'Sin fecha';`

7. Guarde, retire el breakpoint y recargue para verificar que la fecha se visualice correctamente.

Paso 7: Vinculación del Repositorio Remoto en GitHub y Push Final

Una vez finalizado y probado el código, cree el repositorio remoto e integre los cambios:

1. Vaya a [GitHub](#) y cree un nuevo repositorio **público y vacío** llamado `practica-2`. **No** añada README, `.gitignore` o licencias desde la interfaz web.
2. En la terminal de su máquina (en el directorio raíz de `practica-2`), vincule y suba su primer commit en la rama principal `main`:

```
# Renombrar rama a main
git branch -M main

# Vincular con el servidor de GitHub (reemplace SU_USUARIO)
git remote add origin https://github.com/SU_USUARIO/practica-2.git

# Enviar los cambios de la rama main
git push -u origin main
```

3. Simule un flujo de trabajo profesional por ramas. Cree una rama para registrar las correcciones del taller de depuración:

```
# Crear y cambiarse a la rama feature
git checkout -b feature/panel-estudiante

# Agregar todos los archivos al staging
git add .

# Realizar el commit semántico
git commit -m "fix: corregir lectura de dataset y validar"

# Subir la rama feature a GitHub
git push -u origin feature/panel-estudiante
```

4. Realice la integración final fusionando la rama `feature` de vuelta a la rama principal:

```
# Cambiarse a la rama principal
git checkout main

# Fusionar cambios de la feature localmente
git merge feature/panel-estudiante

# Subir cambios integrados a main en GitHub
git push origin main
```

```
# Eliminar la rama local de feature (opcional)
git branch -d feature/panel-estudiante
```

Trampas Comunes

- **Error de Puerto Ocupado Address already in use:** Spring Boot corre por defecto en el puerto 8080. Si el puerto está ocupado por otra aplicación, cree el archivo `application.properties` en `backend/src/main/resources/application.properties` y añada la línea:
`server.port=9090`
- **Exclusividad de Estilos en Shadow DOM:** Los estilos aplicados en el archivo `style.css` global no afectarán el contenido interno de `<tarjeta-asignacion>`. Para estilizar el componente desde el exterior, use **CSS Custom Properties** (variables CSS) declaradas en `:root` que fluyan dentro del Shadow Root.
- **DevTools y la Caché del Navegador:** Los cambios de JavaScript no se refrescan de inmediato si el navegador los almacena en caché. Presione `Ctrl + F5` o mantenga abiertas las DevTools de Chrome para deshabilitar la caché en desarrollo.